

Mission Briefing

You are part of a team responsible for developing the **CPU scheduling system for a large-scale cloud data center**. The system must efficiently manage thousands of processes with different execution behaviors.

Some processes are **interactive and short**, while others are **long-running computational jobs**. To handle this diversity, the operating system uses a **Multi-Level Feedback Queue (MLFQ) Scheduling Algorithm**.

MLFQ dynamically adjusts process priority based on CPU usage. Processes that consume too much CPU time are gradually moved to lower priority queues.

Your task is to implement an **MLFQ Scheduler in C** that reads process information from a **file**, simulates the scheduling behavior, and displays execution results.

System Architecture

The scheduler contains **three priority queues**:

Queue	Priority Level	Scheduling Policy	Time Quantum
Q1	Highest	Round Robin	2 units
Q2	Medium	Round Robin	4 units
Q3	Lowest	FCFS	No Time Quantum

Scheduling Rules

1. All newly arriving processes **enter Queue 1 (Q1)**.
2. If a process **uses its entire time quantum in Q1**, it is **moved to Q2**.
3. If a process **uses its entire time quantum in Q2**, it is **moved to Q3**.
4. If a process **finishes execution before the time quantum expires**, it leaves the system
5. Processes in **Q3 run using FCFS scheduling** until completion.
6. The CPU **always schedules processes from the highest priority non-empty queue**.
7. The scheduler must **check for newly arriving processes after every time slice**.
8. If a process is interrupted 3 times (total interruptions %3) then promote the process to higher queue (if not already in queue 1).
9. If a process is interrupted during execution and has not completed its time quantum place the process at the end of the same queue.

If disrupt flag =0 then the process will be placed at the end with the same quantum time remaining (without counting the executed).

If disrupt flag =1 then process will be placed at the end with the
time quantum-executed time

Example (Quantum = 4):

disrupt_flag = 0: P1 runs 2 ms → disrupted → placed at queue end with **full quantum = 4**. When P1 runs again it gets **4 ms** because the previous execution is ignored.

disrupt_flag = 1: P1 runs 2 ms → disrupted → placed at queue end with **remaining quantum = 2**. When P1 runs again it gets **only 2 ms** because executed time is counted.

10. A process gets full CPU time to execute no matter if it was disrupted or not .(for the specific queue the process is in).
-

Input File Format

The program must read process information from a file named:

processes.txt

Each line in the file represents one process in the following format:

PID ArrivalTime BurstTime disrupt flag

Example file:

```
P1 0 10 0
P2 1 4 1
P3 2 6 1
P4 3 8 0
P5 4 3 1
```

Meaning:

Process	Arrival Time	Burst Time
P1	0	10
P2	1	4
P3	2	6
P4	3	8
P5	4	3

Process Execution Flow

The scheduler must simulate CPU execution step by step.

Example:

Time 0 → P1 arrives and enters Q1

Time 1 → P2 arrives

Time 2 → P3 arrives

Example execution sequence:

Q1 executes P1 for 2 units → moved to Q2

Q1 executes P2 for 2 units → continues or finishes

Q1 executes P3 for 2 units → moved to Q2

...

Program Requirements

1. File Handling

Your program must read process information from **processes.txt** using file handling.

Example:

```
FILE *fp = fopen("processes.txt", "r");
```

Store the processes in an appropriate data structure.

2. Queue Implementation

Implement three ready queues:

Queue 1 → Round Robin (Quantum = 2)

Queue 2 → Round Robin (Quantum = 4)

Queue 3 → FCFS

You may implement queues using:

- Arrays
 - Linked Lists
 - Queue Structures
-

3. CPU Simulation

The program must simulate CPU execution while:

1. Checking for newly arriving processes and starting accordingly. For example if a process arrives at $t=2$ start insert at that time.
 2. Inserting them into **Queue 1**
 3. Selecting processes from the **highest priority non-empty queue**
-

4. Queue Transition Rules

Process movement between queues must follow:

Condition	Action
Process uses full quantum in Q1	Move to Q2
Process uses full quantum in Q2	Move to Q3
Process completes execution	Remove from system
Process in Q3	Run until completion

5. Gantt Chart

Your program must display the **CPU execution timeline**.

Example:

Gantt Chart:

```
0 P1 2
2 P2 4
4 P3 6
6 P1 10
...
```

6. Performance Metrics

For each process calculate:

- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)

Formulas:

Turnaround Time = Completion Time – Arrival Time

Waiting Time = Turnaround Time – Burst Time

7. Final Output Table

Your program should display the results in the following format:

Process	AT	BT	CT	TAT	WT
P1	0	10	25	25	15
P2	1	4	6	5	1
P3	2	6	18	16	10
P4	3	8	30	27	19
P5	4	3	11	7	4

8. Average Statistics

Display:

Average Waiting Time

Average Turnaround Time

Note: You are not allowed to use any builtin function except file reading,input,output.You cannot use builtin data structures .

There can be any number of processes

1 time quantum = 1 iteration of loop (no need to use time library)